Graph Paths II

Consider a directed weighted graph having n nodes and m edges.
Your task is to calculate the minimum path length from node 1 to node n with exactly k edges.

Input

The first input line contains three integers n, m and k: the number of nodes and edges, and the length of the path.
The nodes are numbered 1,2,...,n.

Then, there are m lines describing the edges.
Each line contains three integers a, b and c: there is an edge from node a to node b with weight c.

Output

Print the minimum path length. If there are no such paths, print -1.

Constraints

1 <= n <= 100
1 <= m <= n(n-1)
1 <= k <= 10^9
1 <= a,b <= n
1 <= c <= 10^9

Example

```
Input:
3 4 8
1 2 5
2 3 4
3 1 1
3 2 2

Output:
27
```

DP definition (exactly k edges)

Let:

`dp[t][v] = minimum cost to reach node v using exactly t edges`

This definition is forced by the problem statement.

Base case

At t = 0:

- You are at node 1
- Cost = 0
- All other nodes are unreachable

dp[0][1] = 0
dp[0][v] = INF   (v ≠ 1)

Transition

If there is an edge:

u → v with weight w

Then:

dp[t+1][v] = min(dp[t+1][v], dp[t][u] + w)

This is ordinary shortest-path DP, but with a fixed number of edges.

Why this DP cannot be implemented directly

- t goes up to k
- k ≤ 10^9
- Even one loop over k is impossible

So we ask the key question:

Is the transition rule the same for every step?

Yes.

That means:

- The same computation repeats
- Only the input vector changes

This is the exact condition needed for matrix exponentiation.

Identify the state properly

At step t, the entire DP state is:

$$S(t) = \begin{bmatrix} dp[t][1] \\ dp[t][2] \\ \vdots \\ dp[t][n] \end{bmatrix}$$

It's just putting all node costs into a vector.

Express one DP step using a matrix

We want a matrix M such that:

S(t + 1) = M ⊗ S(t)

Where ⊗ is some kind of multiplication.

One component at a time

Take a fixed node v.

We know from DP:

$$dp[t+1][v] = \min_{u \to v}\big(dp[t][u] + w(u,v)\big)$$

Now rewrite this in matrix language:

$$S(t+1)[v] = \min_{u}\big(S(t)[u] + M[u][v]\big)$$

So naturally:

$$M[u][v] = \begin{cases} w(u,v), & \text{if an edge } u \to v \text{ exists,} \\ \infty, & \text{otherwise.} \end{cases}$$

What kind of matrix multiplication is this?

Normal matrix multiplication:

$$C[i][j] = \sum_{k} A[i][k] \cdot B[k][j]$$

Here, we need:

$$C[i][j] = \min_{k}\big(A[i][k] + B[k][j]\big)$$

This is called min-plus matrix multiplication.

- A[i][k] = cost from i to k
- B[k][j] = cost from k to j
- A[i][k] + B[k][j] = cost of going through k
- min picks the cheapest intermediate node

So matrix multiplication = path concatenation.

What does matrix power represent?

Matrix	Meaning
M^1	minimum cost using 1 edge
M^2	minimum cost using 2 edges
M^3	minimum cost using 3 edges
M^k	minimum cost using k edges

Matrix exponentiation is not about matrices
It is about:

- fixed transitions
- associative composition
- logarithmic jumping

Code

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll INF = (ll)4e18;

struct Matrix {
    int n;
    vector<vector<ll>> a;

    Matrix(int n) : n(n) {
        a.assign(n, vector<ll>(n, INF));
    }
};

// Min-plus matrix multiplication
Matrix multiply(const Matrix &A, const Matrix &B) {
    int n = A.n;
    Matrix C(n);

    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) {
            if (A.a[i][k] == INF) continue;
            for (int j = 0; j < n; j++) {
                if (B.a[k][j] == INF) continue;
                C.a[i][j] = min(C.a[i][j],
                                A.a[i][k] + B.a[k][j]);
            }
        }
    }
    return C;
}

// Fast exponentiation
Matrix matrix_power(Matrix base, long long exp) {
    int n = base.n;
    Matrix result(n);

    // Identity matrix (0 on diagonal, INF elsewhere)
    for (int i = 0; i < n; i++)
        result.a[i][i] = 0;

    while (exp > 0) {
        if (exp & 1)
            result = multiply(result, base);
        base = multiply(base, base);
        exp >>= 1;
    }
    return result;
}

int main() {
    int n, m;
    long long k;
    cin >> n >> m >> k;

    Matrix M(n);

    for (int i = 0; i < m; i++) {
        int u, v;
        ll w;
        cin >> u >> v >> w;
        u--; v--;
        M.a[u][v] = min(M.a[u][v], w);
    }

    Matrix Mk = matrix_power(M, k);

    ll ans = Mk.a[0][n - 1];
    if (ans >= INF / 2) ans = -1;

    cout << ans << "\n";
    return 0;
}
```